FINITE STATE MACHINE

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Reni Soelistijorini, B.Eng., MT

MATA KULIAH: Praktikum Rangkaian Logika 2

TANGGAL: Kamis, 6 Juni 2024

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2024

A. TUJUAN

- Dapat membuat program dengan model disain structural.
- Dapat mengimplementasikan algoritma multiplexer pada disain structural.

B. ALAT DAN BAHAN

1. PC yang sudah terinstal ISE 13.1
2. Xilinx Spartan 3
3. Downloader JTAG USB
4. Power Supply 5 volt

C. Percobaan

1. Traffic Lights 1

a) Program VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity traffic is
port(
clr : in STD_LOGIC;
clk : in STD_LOGIC;
lights : out STD_LOGIC_VECTOR(5 downto 0)
);
end traffic;
architecture Behavioral of traffic is
-- Define states
type state_type is (s0, s1, s2, s3, s4, s5);
signal state : state_type;
-- Define counters and constants
signal count : STD_LOGIC_VECTOR(3 downto 0);
constant SEC5 : STD_LOGIC_VECTOR(3 downto 0) := "1111";
constant SEC1 : STD_LOGIC_VECTOR(3 downto 0) := "0011";
-- Clock divider signals
signal slow_clk : STD_LOGIC := '0';
signal clk_div_count : STD_LOGIC_VECTOR(25 downto 0) := (others => '0');
constant DIV_VALUE : STD_LOGIC_VECTOR(23 downto 0) := X"4C4B40"; --
50,000,000 in hexadecimal (for 1 Hz output with 50 MHz input)
```

```
begin
-- Clock divider process
process(clk)
begin
if rising_edge(clk) then
if clk_div_count = DIV_VALUE then
clk_div_count <= (others => '0');
slow_clk <= not slow_clk;
else
clk_div_count <= clk_div_count + 1;
end if;
end if;
end process;
-- State transition process using slow clock
process (clr, slow_clk)
begin
if clr = '1' then
state <= s0;
count <= X"0";
elsif rising_edge(slow_clk) then
case state is
when s0 =>
if count < SEC5 then
state <= s0;
count <= count + 1;
else
state <= s1;
count <= X"0";
end if;
when s1 =>
if count < SEC1 then
state <= s1;
count <= count + 1;
else
state <= s2;
count <= X"0";
end if;
when s2 =>
if count < SEC1 then
```

```
state <= s2;
count <= count + 1;
else
state <= s3;
count <= X"0";
end if;
when s3 =>
if count < SEC5 then
state <= s3;
count <= count + 1;
else
state <= s4;
count <= X"0";
end if;
when s4 =>
if count < SEC1 then
state <= s4;
count <= count + 1;
else
state <= s5;
count <= X"0";
end if;
when s5 =>
if count < SEC1 then
state <= s5;
count <= count + 1;
else
state <= s0;
count <= X"0";
end if;
when others =>
state <=  s0;
end case;
end if;
end process;
-- Output process for traffic lights
process(state)
begin
case state is
```

```
when s3 => lights <= "001100";
when s4 => lights <= "010100";
when s5 => lights <= "100100";
when others => lights <= "100001";
end case;
end process;
end  Behavioral;
```

b) Test Bench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;
ENTITY maintb IS
END maintb;
ARCHITECTURE behavior OF maintb IS
-- Component Declaration for the Unit Under Test (UUT)
COMPONENT traffic
PORT(
clr : IN std_logic;
clk : IN std_logic;
lights : OUT std_logic_vector(5 downto 0)
);
END COMPONENT;

--Inputs
signal clr : std_logic := '0';
signal clk : std_logic := '0';
--Outputs
signal lights : std_logic_vector(5 downto 0);

BEGIN
-- Instantiate the Unit Under Test (UUT)
uut: traffic PORT MAP (
clr => clr,
clk => clk,
lights => lights
);
```

```
-- Stimulus process
stim_proc: process
begin
-- hold reset state for 100 ns.
wait for 10 ns;

clk <= not clk;

end process;
stim_proc2: process
begin
-- hold reset state for 100 ns.
wait for 200 ns;
clr <= not clr;

end process;
END;
```

2. Traffic Lights 2
   a) Program VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity traffic_empat is
port (
clk : in STD_LOGIC;
rst : in STD_LOGIC;
red_light : out STD_LOGIC;
yellow_light : out STD_LOGIC;
an : out STD_LOGIC;
green_light : out STD_LOGIC
);
end traffic_empat;
architecture Behavioral of traffic_seven is
type state_type is (red, red_yellow, green, yellow);
signal state : state_type;
constant red_duration : natural := 200000000; -- 2 sec
constant red_yellow_duration : natural := 100000000; -- 1 sec
```

```
constant green_duration : natural := 200000000; -- 2 sec
constant yellow_duration : natural := 100000000; -- 1 sec
signal counter : unsigned(31 downto 0) := (others => '0');
begin
an <= '0'; -- Output 'an' always '0' as requested
process (clk, rst)
begin
if rst = '1' then
state <= red;
counter <= (others => '0');
red_light <= '1'; -- Start with red light on
yellow_light <= '0';
green_light <= '0';
elsif rising_edge(clk) then
case state is
when red =>
if counter < red_duration then
red_light <= '1';
yellow_light <= '0';
green_light <= '0';
counter <= counter + 1;
else
state <= red_yellow;
counter <= (others => '0');
end if;
when red_yellow =>
if counter < red_yellow_duration then
red_light <= '1';
yellow_light <= '1';
green_light <= '0';
counter <= counter + 1;
else
state <= green;
counter <= (others => '0');
end if;
when green =>
if counter < green_duration then
```

```
red_light <= '0';
yellow_light <= '0';
green_light <= '1';
counter <= counter + 1;
else
state <= yellow;
counter <= (others => '0');
end if;
when yellow =>
if counter < yellow_duration then
red_light <= '0';
yellow_light <= '1';
green_light <= '0';
counter <= counter + 1;
else
state <= red;
counter <= (others => '0');
end if;
when others =>
null; -- Default case, no action
end case;
end if;
end process;
end  Behavioral;
```

3. Vending Machine
   a) Program Hexseg VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity hex7segb is
Port (
x : in STD_LOGIC_VECTOR (3 downto 0);
a_to_g : out STD_LOGIC_VECTOR (6 downto 0)
);
end hex7segb;
architecture Behavioral of hex7segb is
begin
process(x)
begin
case x is
when "0000" => a_to_g <= "0000001"; -- 0
```

```
when "0001" => a_to_g <= "1001111"; -- 1
when "0010" => a_to_g <= "0010010"; -- 2
when "0011" => a_to_g <= "0000110"; -- 3
when "0100" => a_to_g <= "1001100"; -- 4
when "0101" => a_to_g <= "0100100"; -- 5
when "0110" => a_to_g <= "0100000"; -- 6
when "0111" => a_to_g <= "0001111"; -- 7
when "1000" => a_to_g <= "0000000"; -- 8
when "1001" => a_to_g <= "0000100"; -- 9
when others => a_to_g <= "0111000"; -- F
end case;
end      process;
end Behavioral;
```

b) Program vending VHD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity main is
Port (
btn : in STD_LOGIC_VECTOR (3 downto 0);
reset : in STD_LOGIC;
clk : in STD_LOGIC;
a_to_g : out STD_LOGIC_VECTOR (6 downto 0);
an : out STD_LOGIC_VECTOR (3 downto 0);
dp : out STD_LOGIC
);
end main;
architecture Behavioral of main is
component vending_fsm is
Port (
btn : in STD_LOGIC_VECTOR (3 downto 0);
reset : in STD_LOGIC;
clk : in STD_LOGIC;
total : out STD_LOGIC_VECTOR (9 downto 0)
);
end component;
component hex7segb is
Port (
```

```
x : in STD_LOGIC_VECTOR (3 downto 0);
a_to_g : out STD_LOGIC_VECTOR (6 downto 0)
);
end component;
signal total : STD_LOGIC_VECTOR (9 downto 0);
signal hundreds, tens, unit : STD_LOGIC_VECTOR (3 downto 0);
signal a_to_g_hundreds, a_to_g_tens, a_to_g_units : STD_LOGIC_VECTOR (6
downto
0);
signal digit_select : STD_LOGIC_VECTOR (1 downto 0) := "00"; -- To keep track of
current active digit
begin
U1: vending_fsm
port map (
btn => btn,
reset => reset,
clk => clk,
total => total
);
process(total)
variable total_int : integer;
begin
total_int := to_integer(unsigned(total));
hundreds <= std_logic_vector(to_unsigned(total_int / 100, 4));
tens <= std_logic_vector(to_unsigned((total_int / 10) mod 10, 4));
unit <= std_logic_vector(to_unsigned(total_int mod 10, 4));
end process;
U2: hex7segb
port map (
x => hundreds,
a_to_g => a_to_g_hundreds
);
U3: hex7segb
port map (
x => tens,
a_to_g => a_to_g_tens
);
U4: hex7segb
port map (
x => unit,
a_to_g => a_to_g_units
);
process(clk)
begin
if rising_edge(clk) then
```

```
case digit_select is
when "00" =>
an <= "1110";
a_to_g <= a_to_g_hundreds;
digit_select <= "01";
when "01" =>
an <= "1101";
a_to_g <= a_to_g_tens;
digit_select <= "10";
when "10" =>
an <= "1011";
a_to_g <= a_to_g_units;
digit_select <= "00";
when others =>
an <= "1110";
digit_select <= "01";
end case;
end if;
end process;
dp <= '1';
end Behavioral;
```

D. Data

1. Traffic Light

2. Vending Machine

E. Analisa

Pada traffic light 1, sebuah sistem lampu lalu lintas didesain menggunakan state machine dengan enam kondisi (s0 hingga s5), di mana setiap kondisi mewakili keadaan lampu lalu lintas tertentu. Sistem ini diinisialisasi dengan sinyal reset (clr) dan menggunakan sinyal jam (clk) untuk mengatur transisi antar kondisi. Sebuah clock divider digunakan untuk memperlambat sinyal jam utama menjadi 1 Hz dari input 50 MHz. Transisi antar kondisi dikendalikan oleh penghitung waktu dengan dua durasi tetap, 5 detik (SEC5) dan 1 detik (SEC1), di mana setiap kondisi memiliki durasi yang ditentukan sebelum beralih ke kondisi berikutnya. Output lampu lalu lintas ditentukan berdasarkan kondisi saat ini dalam state machine, dengan masing-masing kondisi mengatur enam bit vektor (lights) yang mengontrol lampu merah, kuning, dan hijau pada dua sisi lalu lintas yang berbeda.

Pada traffic light 2, sebuah sistem lampu lalu lintas didesain dengan empat kondisi: merah (red), merah-kuning (red_yellow), hijau (green), dan kuning (yellow). Sistem ini diatur oleh sinyal reset (rst) dan jam (clk). Setiap kondisi memiliki durasi tertentu: merah selama 2 detik, merah-kuning selama 1 detik, hijau selama 2 detik, dan kuning selama 1 detik. Transisi antar kondisi dikendalikan oleh penghitung (counter) yang bertambah setiap sinyal jam. Saat reset aktif, sistem diinisialisasi ke kondisi merah dengan lampu merah menyala dan lampu lainnya mati. Ketika penghitung mencapai durasi yang ditentukan untuk suatu kondisi, sistem beralih ke kondisi berikutnya, mereset penghitung, dan mengatur lampu lalu lintas sesuai dengan kondisi saat ini. Output 'an' selalu diatur ke '0' sesuai permintaan.

Pada vending machine, sistem dirancang dengan display 7-segmen untuk menampilkan jumlah total yang dimasukkan. Komponen utama terdiri dari state machine vending (vending_fsm) yang menerima input tombol (btn), reset, dan clock (clk), serta mengeluarkan total nilai (total). Nilai total ini kemudian dibagi menjadi ratusan, puluhan, dan satuan yang masing-masing dikonversi ke bentuk 7-segmen melalui komponen hex7segb, yang mengonversi angka biner 4-bit menjadi representasi 7-segmen. Proses utama pada clock menentukan digit yang aktif untuk ditampilkan (hundreds, tens, atau unit) secara bergantian dengan cepat untuk membuat tampilan multiplexing. Output 'an' mengontrol digit yang aktif, sementara 'dp' (decimal point) selalu diatur ke '1', menandakan titik desimal tidak menyala.

F. Kesimpulan

Pada desain sistem lampu lalu lintas traffic light 1, state machine digunakan untuk mengatur enam kondisi lampu dengan durasi yang terkontrol oleh clock divider dan penghitung waktu, memungkinkan transisi yang teratur dan otomatis berdasarkan kondisi yang telah ditentukan. Traffic light 2 menggunakan pendekatan yang lebih sederhana dengan empat kondisi dan durasi tetap untuk setiap kondisi, serta penghitung untuk mengatur transisi antara kondisi, memastikan lampu lalu lintas berfungsi sesuai dengan urutan yang diinginkan. Pada sistem vending machine, state machine mengelola input uang dan menampilkan jumlah total pada display 7-segmen, dengan pengaturan multiplexing untuk menampilkan ratusan, puluhan, dan satuan secara bergantian. Implementasi ini menunjukkan bagaimana state machine dapat digunakan secara efektif untuk mengontrol sistem elektronik yang kompleks dan memastikan operasi yang efisien dan tepat.